

04 — Manual GitHub sync guide (sanitized edition) / 04 — GitHub 手动同步指南（公开脱敏版）

Updated: 2026-09-20

更新日期：2026-09-20

This edition omits internal project names, organization details, and dedicated environment names. The walkthrough and supplied screenshot retain this handbook's local folder path to make the steps concrete; use your own checkout path on another computer. The original internal Markdown and PDF guides remain local and are excluded from Git commits.

本版移除了内部项目名称、组织信息和专用环境名称。为方便对照操作，步骤说明和提供的截图保留了本手册在本机的文件夹路径；在另一台电脑上，请换成自己的仓库路径。原始内部 Markdown 与 PDF 指南仍仅保留本地，不加入 Git 提交。

The example repository is HainanW/research-coding-handbook. GitHub stores files that have been committed and pushed; ignored local originals are not automatically backed up there.

本文以公开仓库 HainanW/research-coding-handbook 为例。GitHub 保存的是已经提交并推送的文件；被忽略的本地原件不会自动备份到 GitHub。

1. Open the repository and check the remote / 1. 进入仓库并确认远程地址 (Repository and remote)

1.1. What does “open PowerShell in the project folder” mean? / 1.1. “在项目文件夹中打开 PowerShell”是什么意思？

It means **open a PowerShell command window and set its current working directory to your project folder**. Git uses that location to find the repository you want to work with. For this handbook, the project folder is research-coding-handbook, containing README.md, docs, and examples.

意思是：**打开 PowerShell 命令窗口，并把它当前工作目录（current working directory）设为项目文件夹**。Git 会根据这个位置找到你要操作的仓库。对于本手册，项目文件夹就是包含 README.md、docs 和 examples 的 research-coding-handbook 文件夹。

In VS Code, follow these steps:

在 VS Code 中，可以这样操作：

1. Select **Terminal** → **New Terminal**. A panel for entering commands opens below the editor. If it uses another shell, select **PowerShell** from the dropdown beside the + button.

点击 **终端（Terminal）** → **新建终端（New Terminal）**。编辑区下方会出现一个可以输入命令的面板。如果当前使用的是其他命令解释器（shell），请在 + 旁的下拉菜单中选择 **PowerShell**。

2. Enter the `cd` command below and press **Enter**. It uses the folder shown in the screenshot; replace the quoted path if your copy is elsewhere.

输入下方的 `cd` 命令，按 **Enter（回车）**。这里使用截图中的文件夹位置；如果你的仓库存放在其他位置，请替换引号内的路径。

3. Enter Get-Location and press **Enter**. Check that its Path output is your research-coding-handbook folder.

输入 `Get-Location`，按回车。检查输出的 `Path` 是否为你的 `research-coding-handbook` 文件夹。

4. Enter `git status` and press **Enter**. It reports the repository's changes; it does not commit or upload files.

输入 `git status`，按回车。它会显示这个仓库的修改状态，不会提交或上传文件。

Enter these commands one at a time in the terminal:

在终端中逐条输入这些命令：

```
cd "C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook"
Get-Location
git status
```

`cd` is an alias for `Set-Location`: it changes the current working directory without moving any files. Quotation marks keep a path containing spaces together. Enter commands in the **terminal panel**, not in a `.py` or `.md` file. The leading `ps ...>` is PowerShell's prompt; you do not type it yourself.

`cd` 是 `Set-Location` 的别名 (alias)，表示切换当前工作目录，不会移动任何文件。引号可以让包含空格的路径仍被当作一个整体。这些命令都输入在**终端面板**里，不是写进 `.py` 或 `.md` 文件。行首的 `PS ...>` 是 PowerShell 自动显示的提示符 (prompt)，不需要自己输入。

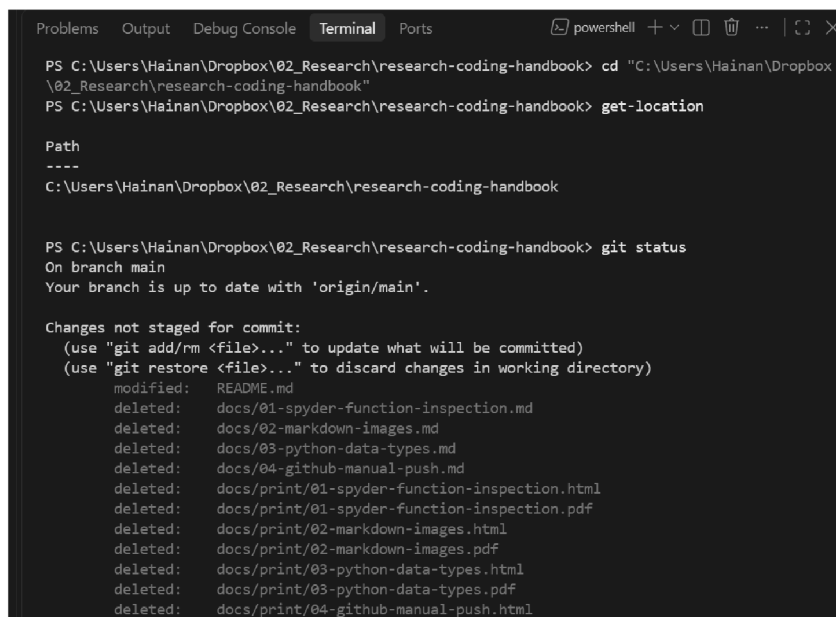


Figure 1. Change to the project folder with `cd`, confirm the location with `Get-Location`, and inspect changes with `git status`. This is the state when the screenshot was taken; your file list may differ. [View the full-size screenshot.](#)

图 1: 用 `cd` 切换到项目文件夹, 用 `Get-Location` 核对位置, 再用 `git status` 查看改动。这是截图时的状态, 你看到的文件列表可能不同。查看原尺寸截图。

Read the screenshot as follows:

可以这样理解截图中的信息：

- Path confirms the project directory, and On branch main identifies the current branch.

Path 用于核对项目目录, On branch main 表示当前分支是 main。

- Changes not staged for commit means there are changes that have not been staged. up to date with 'origin/main' compares commits with the locally recorded remote branch; it does not mean the working folder has no changes or that Git has just checked GitHub.

Changes not staged for commit 表示还有未暂存的改动。up to date with 'origin/main' 比较的是当前提交与本地记录的远程分支，并不表示工作区没有改动，也不代表刚刚核对过 GitHub 上的最新状态。

- After this handbook's filename changes, old tracked paths appear as deleted, while new paths may appear under Untracked files farther down the full output. Check that the new NN-main-... files exist and review the complete list; an old path marked deleted alone does not mean its content was lost.

本手册更改文件名后，已跟踪的旧路径显示为 deleted，新路径可能显示在完整输出后面的 Untracked files（未跟踪文件）中。请确认新的 NN-main-... 文件存在，并查看完整清单；仅凭旧路径标为 deleted，不能认定文件内容丢失。

Official references: [VS Code terminal basics](#) ^[1], [PowerShell current working directory](#) ^[2], and [Git status](#) ^[3].

官方资料: [VS Code 终端基础](#) ^[1]、[PowerShell 当前工作目录](#) ^[2]，以及 [Git status](#) ^[3]。

1.2. Check the repository and remote / 1.2. 检查仓库与远程地址

Once PowerShell is in the project folder, run these commands individually:

确认 PowerShell 已定位到项目文件夹后，逐条执行：

```
git status
git remote -v
git branch --show-current
```

For this repository, origin should point to <https://github.com/HainanW/research-coding-handbook.git>, and the branch is main.

本仓库的 origin 应指向 <https://github.com/HainanW/research-coding-handbook.git>，当前分支为 main。

1.3. First download on another computer / 1.3. 在另一台电脑首次下载

On another computer, download the repository from its intended parent directory:

在另一台电脑首次下载时，可以在目标父目录运行：

```
git clone https://github.com/HainanW/research-coding-handbook.git
cd research-coding-handbook
```

The clone destination must not exist or must be empty. If a directory already contains files, compare, back up, and reconcile them first instead of overwriting them.

git clone 的目标目录应不存在或为空。已有文件的文件夹需要先比较、备份并合并文件，不能直接覆盖。

2. Get updates from GitHub / 2. 获取 GitHub 上的更新 (Pull)

With a clean working tree, run:

工作区干净时执行：

```
git pull --ff-only origin main
```

If you have uncommitted changes, review and commit them using the next section first. If the branches have diverged, inspect `git log --oneline --graph --all` and reconcile the histories before pushing. Do not forcibly overwrite remote history.

如果有尚未提交的修改，先按下一节检查并提交，再拉取。如果提示分支已经分叉，先检查 `git log --oneline --graph --all`，协调合并后再推送。不要强制覆盖远程历史。

2.1. What does `git pull --ff-only` mean? / 2.1. `git pull --ff-only` 是什么意思?

`--ff-only` means **fast-forward only (allow only fast-forward updates)**.

`--ff-only` 是 **fast-forward only (只允许快进更新)**。

```
git pull --ff-only
```

This means: fetch remote updates, and **update the local branch only when it can move directly forward along the current commit history**.

意思是：获取远程更新，**只有能沿着当前提交历史直接往前走时，才更新本地分支**。

For example, the local branch is simply behind the remote branch:

例如，本地只是落后于远程：

```
本地: A → B
远程: A → B → C → D
```

It can update directly to D. This is called a “fast-forward.”

可以直接更新到 D，这就叫“快进”。

If the local and remote branches each have different new commits:

如果本地和远程各自有了不同的新提交：

```
本地: A → B → E
远程: A → B → C → D
```

The histories have diverged. The command reports an error and stops integration, leaving you to decide how to handle it.

历史已经分叉，命令会报错并停止整合，留给你决定如何处理。

It suits the routine synchronization in the guide: **it avoids automatically creating a merge commit during a pull**. `ff` does not mean “force overwrite.”

它适合教程里的日常同步：**避免拉取时自动产生合并提交 (merge commit)**。`ff` 不是“强制覆盖”的意思。

Further Details / 补充说明

The letters in these diagrams represent example commits, not this repository's actual history. In the first example, moving the local branch from B to D uses the existing commits C and D; it does not create a new merge commit. In the second example, the local commit E is on a different path from C and D, so advancing along a single history is not possible.

示意图里的字母表示教学示例中的提交，不是本仓库的实际历史。第一个例子中，本地分支从 B 推进到 D，直接使用已有的提交 C 和 D，不会创建新的合并提交。第二个例子中，本地提交 E 与 C、D 处于不同的路径，因此无法沿着同一条历史直接前进。

`git pull` first fetches updates from the configured upstream branch, then tries to integrate them into the current branch. If `--ff-only` rejects the integration in the second example, the current local branch stays at E. The fetch step may already have downloaded remote commits and updated remote-tracking branches such as `origin/main`; the error does not mean that nothing was fetched.

`git pull` 先从配置好的上游分支 (upstream branch) 获取更新 (fetch)，再尝试整合到当前分支。在第二个例子中，如果 `--ff-only` 拒绝整合，当前本地分支仍停在 E。前面的 fetch 步骤可能已经下载了远程提交，并更新了 `origin/main` 等远程跟踪分支 (remote-tracking branches)；报错并不意味着没有获取到任何更新。

The command at the start of section 2 includes `origin main: origin` selects the remote repository, and `main` selects its branch. The shorter `git pull --ff-only` uses the current local branch's configured upstream. If your current local branch is `main` and its upstream is `origin/main`, both commands fetch from the same branch and require fast-forward integration. Neither command switches your current branch.

第 2 节开头的命令带有 `origin main: origin` 指定远程仓库，`main` 指定该远程仓库中的分支。简写 `git pull --ff-only` 使用当前本地分支配置好的上游分支。如果当前本地分支为 `main`，上游为 `origin/main`，这两种写法获取的是同一分支的更新，并且都只允许快进整合。两条命令都不会替你切换当前分支。

Source: [Git documentation — git pull, including --ff-only](#) ^[4].

来源：Git 官方文档：[git pull](#)，含 `--ff-only` 选项说明 ^[4]。

3. Review, stage, and commit local files / 3. 检查、暂存并提交本地文件 (Review, stage, commit)

```
git status --short
git diff
git add -- README.md
git diff --cached --stat
git diff --cached
git commit -m "Update handbook documentation"
```

Replace `README.md` with the files intended for this upload; multiple paths may be listed. Describe the actual change in the commit message. Open new files to review them, and inspect binary documents such as Word and PDF files rather than relying only on the diff summary.

将 `README.md` 替换为本次确实需要上传的文件名，也可以一次列出多个文件。提交说明应描述实际修改。新文件需要单独打开检查；Word、PDF 等二进制文件也要查看内容，不能只看差异摘要。

Before publishing, check for passwords, tokens, internal addresses, or other material that should not be public. The two original `GitHub_Manual_Push_Guide` files at the repository root are excluded by `.gitignore`; do not force them into Git with `git add -f`.

公开上传前，确认文件不含密码、令牌、内部地址或不应公开的资料。本仓库根目录下的两份 `GitHub_Manual_Push_Guide` 原件已经由 `.gitignore` 排除，不要使用 `git add -f` 强制加入。

If Git reports a missing author identity, configure your author name and verified email, or your GitHub-provided private email, for this repository. There is no need to change global settings for other projects.

若 Git 提示没有作者身份，可仅为当前仓库配置自己的 GitHub 用户名和已验证邮箱，或使用 GitHub 设置中提供的隐私邮箱。无需更改其他项目的全局设置。

4. Push to GitHub / 4. 推送到 GitHub (Push)

```
git pull --ff-only origin main
git push origin main
```

Only continue after each command succeeds. Resolve push errors instead of using `--force`.

每条命令成功后再执行下一条。推送失败时先处理错误，不要使用 `--force`。

If you have a separately approved backup repository, configure and push to that remote separately. An editor's sync action does not necessarily update every remote. This example uses only `origin`.

如果另有经过确认的第二个备份仓库，需要单独配置远程并分别推送；一次编辑器“同步”操作不保证更新所有远程。本示例只配置 `origin`。

5. Verify synchronization / 5. 核对同步结果 (Verification)

```
git status
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

There should be no uncommitted changes, and the full local HEAD hash should match the remote `main` hash. A clean working tree alone does not prove your commits have been uploaded. Ignored files stay local and do not appear in ordinary `git status` output.

工作区应没有待提交修改，本地 HEAD 与远程 `main` 的完整提交编号应一致。工作区干净本身不代表已经上传。被忽略的文件仍保留本地，不会出现在普通 `git status` 中。

6. Directory ownership errors / 6. 目录所有权错误 (Dubious ownership)

If Git reports detected dubious ownership, first verify that the current directory is a local repository you trust, then run:

如果 Git 提示 detected dubious ownership，先确认当前目录确实是你信任的本地仓库，再执行：

```
$TrustedRepo = (Get-Location).Path -replace '\\', '/'
git config --global --add safe.directory $TrustedRepo
```

This adds an exception for the specific verified directory. Do not set `safe.directory` to `*`.

这只为当前已确认的目录添加例外。不要将 `safe.directory` 设置为 `*`。

7. Troubleshooting / 7. 常见问题 (Troubleshooting)

- non-fast-forward: the remote has commits you have not integrated. Fetch, inspect the differences, reconcile the histories, and retry.

non-fast-forward: 远程包含本地尚未整合的提交。先获取并检查差异，处理合并后重试。

- Conflicting files: retain both versions for comparison. Binary files often require a manual choice or separate copies.

同名文件冲突：保留两份内容进行比较；二进制文件通常需要人工选择或分别保存。

- Missing uploads: check whether the files were staged, committed, and pushed, and whether .gitignore excludes them.

文件没有上传：检查是否已经暂存、提交和推送，以及是否匹配 .gitignore。

- Sensitive content committed by mistake: .gitignore does not remove existing history. Address exposed credentials first, then determine the appropriate history cleanup.

误提交敏感资料：.gitignore 不会删除已有历史；先处理已暴露的凭据，再按具体情况清理历史。

Daily sequence: pull updates → edit → review and commit → push → verify commit hashes.

日常流程是：拉取更新 → 修改文件 → 检查并提交 → 推送 → 核对提交编号。

参考资料 (References)

1. <https://code.visualstudio.com/docs/terminal/basics>
2. <https://learn.microsoft.com/en-us/powershell/scripting/samples/managing-current-location>
3. <https://git-scm.com/docs/git-status>
4. <https://git-scm.com/docs/git-pull>